

CURATOR

Cost-Aware Environment Selection for Agentic Reinforcement Learning Training of Large Language Models

Name	Roll No.
Nishant Shah	23166
Ansh Bajpai	23108
M. Ruthik Reddy	23129

1. Problem Background

Reinforcement learning (RL) has recently gained traction with respect to post-training LLMs. Approaches such as GRPO, widely used in DeepSeek Math and DeepSeek-R1, involve training models on problems that provide evaluable rewards such as in mathematics, programs with unit tests, logic problems, and instances of tool use. Given that LLMs have recently come to be used as agents, the training relies on a collection of diverse RL environments as opposed to a single dataset. The above transition creates a problem of resource allocation. As environments differ in the implication for agent training, they share the necessity of consuming the finite resource of GPU computing power in terms of rollout generation, reward acquisition, and policy updating.

- **Saturation:** easy environments stop producing learning signal once the model solves them reliably.
- **Zero signal:** environments that are too hard yield all-fail rollout groups, which give GRPO no gradient.
- **Unequal cost:** a code environment that runs unit tests or a multi-turn tool-use environment can cost several times more per rollout than a short math prompt.
- **Interference and transfer:** training on one environment can help or hurt performance on others.

In practice, the training mixture is usually fixed by hand or sampled uniformly, and it is rarely revisited during a run. Teams therefore spend compute on environments that no longer help, and after training they cannot say which environments were worth their cost.

2. Problem Statement

Given a set of N training environments, a held-out evaluation benchmark and a fixed compute budget B , decide online which environments an LLM agent should train on at each step so as to maximise held-out benchmark improvement per unit of compute (GPU per dollar), while the value of each environment changes as the agent learns.

The system must also produce, at the end of training, a validated return-on-investment (ROI) ranking of environments that states which environments contributed real gains and which wasted compute.

3. Why Reinforcement Learning?

Curator involves reinforcement learning at two levels. The inner level is the RL post-training of the LLM agent itself. The outer level, the environment scheduler, is a sequential decision-making problem that fits the bandit branch of RL rather than supervised learning or static optimisation, for the following reasons.

Property of the problem	Why it requires RL / bandits
Sequential decisions under uncertainty	The scheduler repeatedly chooses a training mixture and only learns an environment's value by training on it.
Exploration vs exploitation	Over-committing to a currently strong environment risks missing a slower but larger payoff; bandit algorithms balance this explicitly.
Partial (bandit) feedback	Gains are observed only for environments actually selected, so there are no labelled targets for supervised learning.
Non-stationary rewards	An environment's value falls as the agent masters it; discounted or sliding-window bandits and EXP3 are designed for drifting rewards.
Delayed, noisy outcomes	Benchmark gains arrive late and aggregate many updates, which calls for online credit assignment rather than one-shot optimisation.
No offline dataset of good mixtures	The optimal mixture depends on the model, environments and budget, so it must be learned during training.

A static search such as grid search over mixtures would require many full training runs, which defeats the purpose of saving compute. An online bandit learns the allocation within a single run.

4. Literature Review

Paper	Main Contribution	Limitation for Curator
Graves et al. (2017), Automated Curriculum Learning for Neural Networks Link	It introduced an automated curriculum in which a bandit dynamically selects training tasks according to learning progress. It also demonstrated faster learning on LSTM-based sequence-learning tasks.	It does not explicitly account for compute cost and was evaluated on neural-network sequence-learning tasks rather than LLM RL environments.
Matiisen et al. (2020), Teacher-Student Curriculum Learning (TSCL) Link	It introduced a teacher that automatically selects subtasks where the student is making rapid progress, while revisiting tasks where performance deteriorates. Moreover it demonstrated results on LSTM addition and Minecraft navigation.	Does not model per-task compute cost and does not provide validated attribution of how much each task contributed to final performance.
Wang et al. (2025), DUMP: Automated Distribution-Level Curriculum Learning for RL-based LLM Post-training Link	It extends curriculum learning to RL-based LLM post-training by dynamically adjusting sampling probabilities across heterogeneous data distributions. It explicitly demonstrates distribution-level adaptive scheduling with GRPO.	It optimises learnability, rather than explicitly optimising benchmark improvement per unit of measured compute; no environment-level ROI attribution.
Chen et al. (2025), Self-Evolving Curriculum for LLM Reasoning (SEC) Link	The main concept of the paper is that it learns the curriculum concurrently with RL fine-tuning, treating problem categories such as difficulty or problem type as bandit arms. It also demonstrated improved reasoning and skill balance across planning, inductive reasoning and mathematics.	It uses a proxy reward without periodic benchmark calibration and does not explicitly model compute cost or environment-level ROI.

Paper	Main Contribution	Limitation for Curator
Gao et al. (2025), Prompt Curriculum Learning for Efficient LLM Post-Training (PCL) Link	It introduces efficient prompt-level curriculum learning by using a concurrently trained value model to identify informative, intermediate-difficulty prompts without costly rollout-based filtering.	It only operates at the prompt level, not environment level, and does not provide environment-level compute-cost accounting or contribution/ROI attribution.
Gu et al. (2026) Actor-Curator: Co-adaptive Curriculum Learning via Policy-Improvement Bandits for Scalable RL Post-training Link	It introduces a learned neural curator that directly optimises policy improvement for adaptive problem selection, providing a theoretically grounded bandit-based curriculum for scalable LLM RL post-training.	Operates primarily at the individual-problem level and focuses on policy improvement rather than explicitly optimising benchmark improvement per measured GPU-dollar. It does not provide Curator's proposed environment-level cost accounting or validated environment-level ROI attribution through leave-one-out analysis.

5. Existing Systems

System	Type	What it provides	What it lacks for Curator
Hugging Face TRL	Open-source RL/post-training library	Provides trainers and tools for methods including GRPO, PPO-related workflows, DPO, reward modeling and SFT, integrated with the Transformers ecosystem.	Provides the training infrastructure but does not provide Curator's environment-level cost-aware scheduler or validated environment ROI attribution.
verl (ByteDance)	Open-source LLM RL training framework	Provides scalable RL post-training based on HybridFlow, with integrations including PyTorch FSDP, Megatron-LM, vLLM and SGLang. It also supports agentic RL, multi-turn conversations and tool calls.	Provides flexible RL execution and infrastructure but does not provide Curator's cost-aware environment selection and environment-level ROI analysis.

System	Type	What it provides	What it lacks for Curator
OpenRLHF	Open-source distributed RL framework	Provides distributed RLHF/RL training infrastructure for LLMs, including integration with distributed training and inference systems.	Does not provide Curator's dynamic environment portfolio optimisation based on benchmark gain per GPU-dollar.
Prime Intellect Environments Hub / Verifiers	RL environment ecosystem	Provides reusable RL environments, evaluation infrastructure and standardised environment/verifier interfaces. Prime Intellect describes the Hub as a community platform for discovering and sharing environments for RL training and evaluation.	Provides and manages environments, but Curator's proposed contribution is the adaptive selection and compute allocation among environments based on measured utility and cost.
Reasoning Gym	Procedural RL environment library	Provides procedurally generated, algorithmically verifiable reasoning environments across mathematics, logic, games, geometry, graphs and other domains.	Provides environments and verification, but not a cost-aware scheduler or environment-level ROI attribution system.
OpenAI Reinforcement Fine-Tuning	Managed RL fine-tuning service	Provides hosted reinforcement fine-tuning with configurable graders for supported models/workflows.	Does not expose Curator's proposed multi-environment optimisation and environment-level ROI attribution workflow.
Weights & Biases / MLflow	Experiment tracking and monitoring	Provides experiment logging, metric tracking, dashboards, run comparison and artifact/metadata management. MLflow, for example, tracks parameters, metrics, code versions and artifacts across runs.	Primarily provides monitoring and analysis rather than making the training-environment allocation decision or validating environment-level causal contribution.

6. Gap Analysis

Gap	Evidence from literature and systems	How Curator addresses it
Compute cost is ignored	All six core papers optimise gain per step or learnability; CERO budgets rollouts per prompt only; trainers track no per-environment cost	Reward is benchmark gain divided by measured GPU-dollar cost per environment
Proxy signals are not grounded	SEC and DUMP rely on absolute advantage; PCL on predicted success rate. BMC-T and GradAlign add target utility, but only at prompt level	Two-speed signal: proxy rewards every step, re-calibrated against held-out benchmark gains every K round
No validated attribution	No prior method checks whether its selections caused the gains	ROI leaderboard compared with leave-one-out ablations using Spearman rank correlation
Selection granularity mismatch	Most recent work selects prompts or problems; agentic training is organised by environments with different costs and interfaces	Arms are whole environments, matching how teams build and budget agent training
Poor practitioner tooling	Methods ship as research code; trackers only monitor	Plug-in sampler for TRL and verl plus a dashboard with live weights, ROI ranking and alerts
Benchmark overfitting risk	Optimising directly on evaluation data can leak into training decisions	Separate untouched test set reserved for final reporting

7. Proposed Research Direction

We propose Curator, an environment-portfolio optimiser that treats training-environment selection for LLM agent RL as a cost-aware, non-stationary bandit problem.

7.1 Formulation

- Arms: the N training environments.
- Proxy reward (every round): $\tilde{r}_i = (\alpha \cdot \text{learning progress}_i + \beta \cdot \text{signal richness}_i) / \text{cost}_i$, where signal richness is the fraction of prompts with pass rate between 0.1 and 0.9.
- Calibrated reward (every K rounds): held-out benchmark gain ΔS credited to each environment in proportion to its compute share, divided by its cost; α and β are re-fitted so proxies track real gains.
- Scheduler: discounted UCB, $UCB_i = \mu_i(\gamma) + c \cdot \sqrt{(\log T / n_i)}$, converted to mixture weights $w_i = (1-\epsilon) \cdot \text{softmax}(UCB_i / \tau) + \epsilon / N$. Sliding-window UCB, EXP3 and discounted Thompson sampling are compared as variants.
- Saturation handling: environments with sustained pass rate above 0.9 or near-zero learning progress are down-weighted.

7.2 System Components

Environment Registry, Cost Meter, Trainer Hook (TRL first, then verl), Signal Engine, Evaluator, Bandit Scheduler, Attribution and ROI Engine, and Dashboard.

7.3 Experimental Plan

- Models: Qwen2.5-0.5B-Instruct for development and Qwen2.5-1.5B-Instruct for main results, trained with GRPO.
- Environments: GSM8K-style math, MATH subset, Countdown, code with unit tests, Knights-and-Knaves logic, tool use, plus a noisy-reward environment and a too-hard environment as sanity checks.
- Baselines: uniform sampling, size-proportional sampling, fixed hand-tuned mixture, learning-progress curriculum, SEC-style absolute-advantage bandit, and ablations of Curator without cost normalisation and without benchmark calibration.
- Metrics: benchmark score versus GPU-hours, compute to reach a target score, area under the score-cost curve, final untouched test score, Spearman ρ of the ROI leaderboard against leave-one-out, and Curator's own overhead.

7.4 Expected Outcomes

1. A cost-normalised bandit formulation of environment selection for agentic LLM RL.

2. Evidence that Curator matches uniform sampling's final score at lower compute, averaged over three seeds.
3. A validated ROI leaderboard showing which environments were worth their compute.
4. An open-source plug-in and dashboard for existing RL trainers.